

Capstone Project – Environmental Data Analytics (Team 14)

Week 3: Automated Data Pipeline & Advanced Storage Architecture

1. Project Objective

The objective of Week 3 is to design and implement an automated, scalable, and production-ready data pipeline to handle environmental datasets efficiently.

This phase builds upon:

- Week 1: Data collection and exploratory data analysis
- Week 2: Data cleaning, transformation, and integration

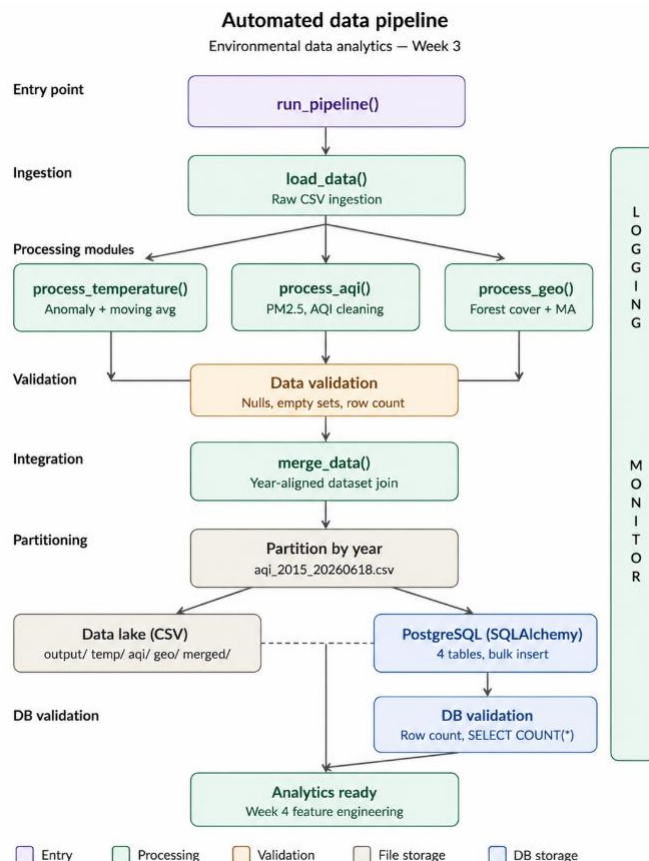
The goal was to convert these processes into a robust, automated pipeline system capable of ingestion, transformation, storage, and monitoring.

2. Pipeline Architecture Overview

The system follows a modern data engineering architecture:

Raw Data → Ingestion → Cleaning → Transformation → Aggregation →

Partitioning → Data Lake (CSV) → PostgreSQL Database → Analytics Ready



The pipeline integrates both file-based storage(Data lake) and relational database storage, forming a hybrid architecture.

3. Work Completed

✓ 3.1 Automated Data Pipeline

A fully automated pipeline was developed using Python to execute all preprocessing steps from Week 2.

Key Functionalities:

- Data ingestion from raw datasets
- Data cleaning and preprocessing
- Feature extraction (Year, Month)
- Data transformation
- Aggregation (daily → monthly → yearly)
- Dataset alignment and merging

The pipeline is executed through a single entry point: `run_pipeline()`

👉 This ensures end-to-end automation

✓ 3.2 Modular Pipeline Design

The pipeline was modularized into reusable components:

- `load_data()` – Data ingestion
- `process_temperature()` – Temperature processing
- `process_aqi()` – Air quality processing
- `process_geo()` – Geospatial processing
- `merge_data()` – Dataset integration
- `save_to_postgres()` – Database storage

👉 Benefits:

- Reusability
- Maintainability
- Clear structure

✓ 3.3 Configuration-Driven Design

A configuration file (`config.json`) was introduced to manage:

- Input data paths
- Output storage paths
- Database connection details

👉 Eliminates hardcoding and improves flexibility.

✓ 3.4 Logging & Monitoring

A logging system (pipeline.log) was implemented to track:

- Execution steps
- Data processing status
- Errors and issues

✓ 3.5 Data Validation (Advanced Feature)

To ensure data quality, validation checks were implemented:

- Detection of empty datasets
- Identification of missing values

👉 Enhances pipeline reliability and robustness.

✓ 3.6 Metadata Tracking (Advanced Feature)

Pipeline logs include metadata such as:

- Number of records processed
- Number of rows in each dataset
- Final merged dataset size

👉 Provides visibility into pipeline performance.

✓ 3.7 Data Lake Storage

Processed data is stored in a structured file-based data lake:

output/

└─ temperature/

└─ aqi/

└─ geo/

└─ merged/

Each directory represents a logical dataset category.

✓ 3.8 Data Partitioning

Datasets are partitioned based on Year:

aqi_2015_20260618.csv

geo_2005_20260618.csv

👉 Benefits:

- Faster data access

- Efficient storage
- Scalable design

✓ 3.9 Versioned Data Storage

All outputs include timestamps:

merged_20260618_120501.csv

👉 Enables:

- Data version control
- Traceability
- Reproducibility

✓ 3.10 PostgreSQL Database Integration

The pipeline integrates PostgreSQL for structured data storage.

Tables Created:

- temperature_data
- air_quality
- geospatial
- environment_merged

Implementation:

- Used SQLAlchemy for efficient bulk data insertion
- Automatic table creation
- Data written directly from processed DataFrames

👉 Benefits:

- Structured querying (SQL)
- Persistent storage
- Industry-standard approach

✓ 3.11 Database Validation (Advanced Feature)

After insertion, validation checks are performed:

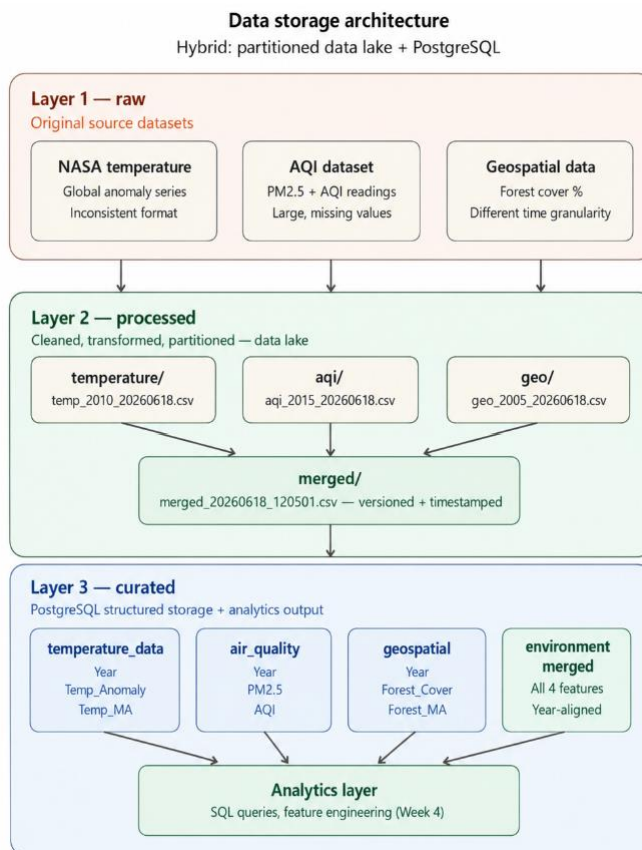
- Row count verification
- Confirmation of successful data loading

Example: `SELECT COUNT(*) FROM environment_merged;`

👉 Ensures data integrity.

4. Data Storage Architecture

A hybrid architecture was implemented:



✓ Layered Architecture

- Raw Layer → Original datasets
- Processed Layer → Cleaned and transformed data
- Curated Layer → Integrated dataset

5. Schema Design

◆ Temperature Table

- Year
- Temperature_Anomaly
- Temp_MA

◆ Air Quality Table

- Year
- PM2.5
- AQI

◆ Geospatial Table

- Year

- Forest_Cover
- Forest_MA

◆ Integrated Table

- Year
- Temperature_Anomaly
- PM2.5
- AQI
- Forest_Cover

Table	Records
temperature_data	26
air_quality	18
Geospatial	3
environment_merged	1

6. Challenges Faced

- Handling missing values in large AQI dataset
- Cleaning NASA dataset with inconsistent format
- Aligning datasets with different time granularities
- Limited overlapping years for integration
- Designing scalable storage architecture
- Ensuring reliability of automated pipeline

7. Key Enhancements Over Week 2

Week 2	Week 3
Manual processing	✓ Automated pipeline
Basic storage	✓ Data lake + PostgreSQL
No validation	✓ Data validation
No monitoring	✓ Logging + metadata
Static outputs	✓ Versioned outputs
No partitioning	✓ Time-based partitioning

8. Alignment with Requirements

Requirement	Status
Automated pipeline	✓ Completed
Data ingestion	✓ Completed
Partitioning	✓ Implemented
Database/storage	✓ PostgreSQL + Data Lake
Deliverables	✓ Pipeline code + schema

9. Future Scalability

The pipeline can be extended to:

- Cloud storage (AWS S3, Azure Data Lake)
- Distributed systems (Apache Spark)
- Workflow tools (Apache Airflow)
- Advanced modeling pipelines

10. Conclusion

Week 3 successfully transformed the project into a fully automated, scalable, and production-ready data pipeline system.

Key achievements:

- Automated data processing
- Structured data storage
- Database integration
- Monitoring and validation
- Scalable architecture
- The pipeline successfully converts raw environmental datasets into analytics-ready datasets through automated ingestion, validation, partitioning, storage, and monitoring processes.
- The three datasets originate from different sources and different time granularities. After aligning on common years, only one overlapping year was available. The pipeline itself is scalable and will automatically integrate additional years when more aligned data becomes available.
- This implementation simulates a real-world data engineering pipeline combining automation, storage design, and database integration.

The system is now fully prepared for advanced feature engineering and analytics in Week 4.

11. Project Links

GitHub: <https://github.com/VahidaSS/CapstoneProject>

Google Drive:

https://drive.google.com/drive/folders/15FRK8ErVL9vvUqc0ocuAngPXZdyHZ7h2?usp=drive_link

12. Screenshots

Figure 1: Pipeline Execution

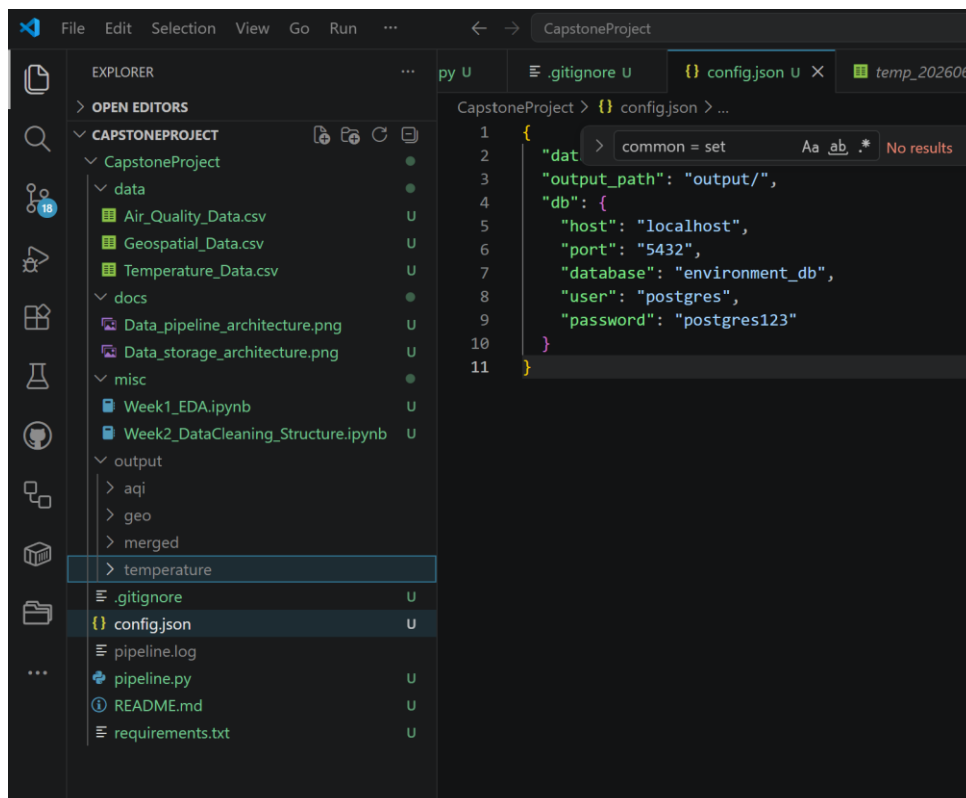
```
kva\HPH435YU@MINGW64 ~/CapstoneProject/CapstoneProject (main)
$ pip install -r requirements.txt
Requirement already satisfied: pandas in c:\users\kva\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from -r requirements.txt (line 1)) (3.0.3)
Collecting sqlalchemy (from -r requirements.txt (line 2))
  Downloading sqlalchemy-2.0.51-cp314-cp314-win_and64.whl.metadata (9.8 kB)
Collecting psycopg2-binary (from -r requirements.txt (line 3))
  Downloading psycopg2-binary-2.9.12-cp314-cp314-win_and64.whl.metadata (5.1 kB)
Requirement already satisfied: numpy>=2.3.3 in c:\users\kva\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from pandas->-r requirements.txt (line 1)) (2.4.6)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\kva\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from pandas->-r requirements.txt (line 1)) (2.9.0.post0)
Requirement already satisfied: tzdata in c:\users\kva\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from pandas->-r requirements.txt (line 1)) (2026.2)
Collecting greenlet==1 (from sqlalchemy->-r requirements.txt (line 2))
  Downloading greenlet-3.5.2-cp314-cp314-win_and64.whl.metadata (3.9 kB)
WARNING: Cache entry deserialization failed, entry ignored
Collecting typing-extensions>=4.6.0 (from sqlalchemy->-r requirements.txt (line 2))
  Using cached typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: sio>=1.5 in c:\users\kva\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from python-dateutil->2.8.2->pandas->-r requirements.txt (line 1)) (1.17.0)
  Downloading sqlalchemy-2.0.51-cp314-cp314-win_and64.whl (2.1 MB)
    2.1/2.1 MB 15.3 MB/s 0:00:00
  Downloading psycopg2-binary-2.9.12-cp314-cp314-win_and64.whl (2.8 MB)
    2.8/2.8 MB 24.6 MB/s 0:00:00
  Downloading greenlet-3.5.2-cp314-cp314-win_and64.whl (240 kB)
Using cached typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Installing collected packages: typing-extensions, psycopg2-binary, greenlet, sqlalchemy
Successfully installed greenlet-3.5.2 psycopg2-binary-2.9.12 sqlalchemy-2.0.51 typing-extensions-4.15.0

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: c:\users\kva\appdata\local\python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

kva\HPH435YU@MINGW64 ~/CapstoneProject/CapstoneProject (main)
$ python pipeline.py
Pipeline started...
Pipeline executed successfully!
```

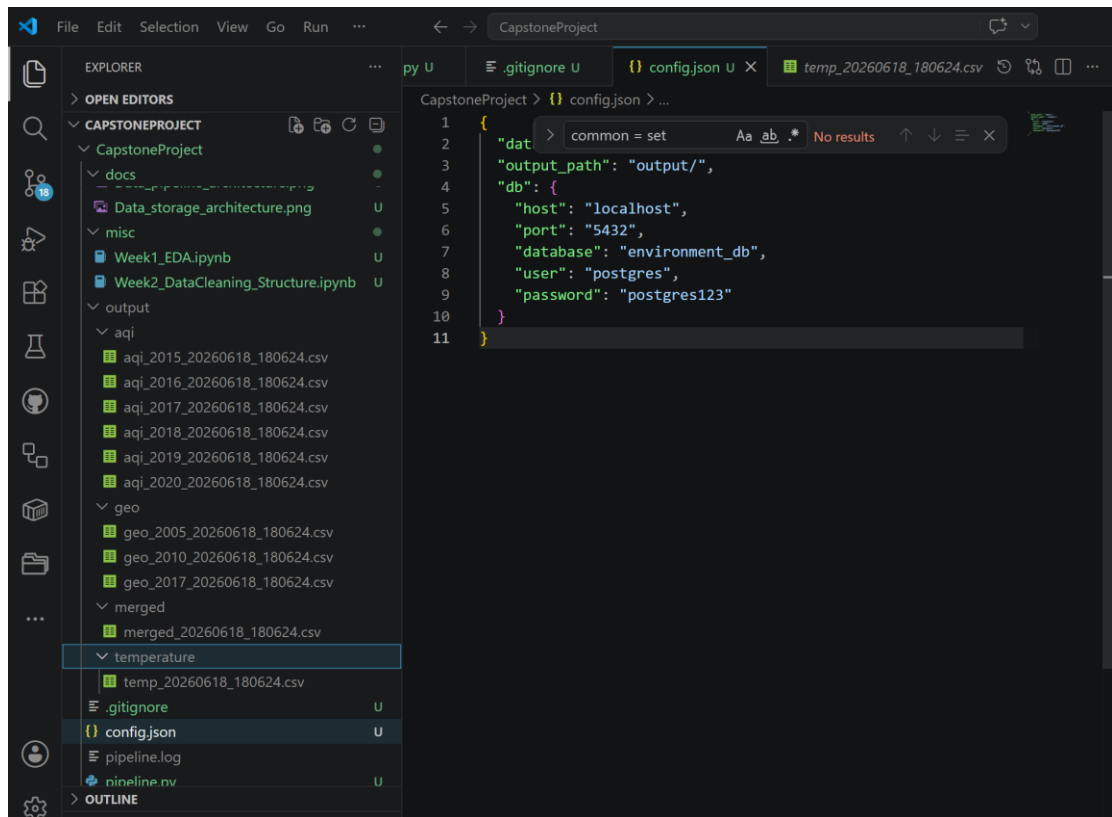
This screenshot shows the successful execution of the pipeline using the command "python pipeline.py". It confirms that the data ingestion, processing, and storage operations were completed without errors.

Figure 2: Output Folder Structure



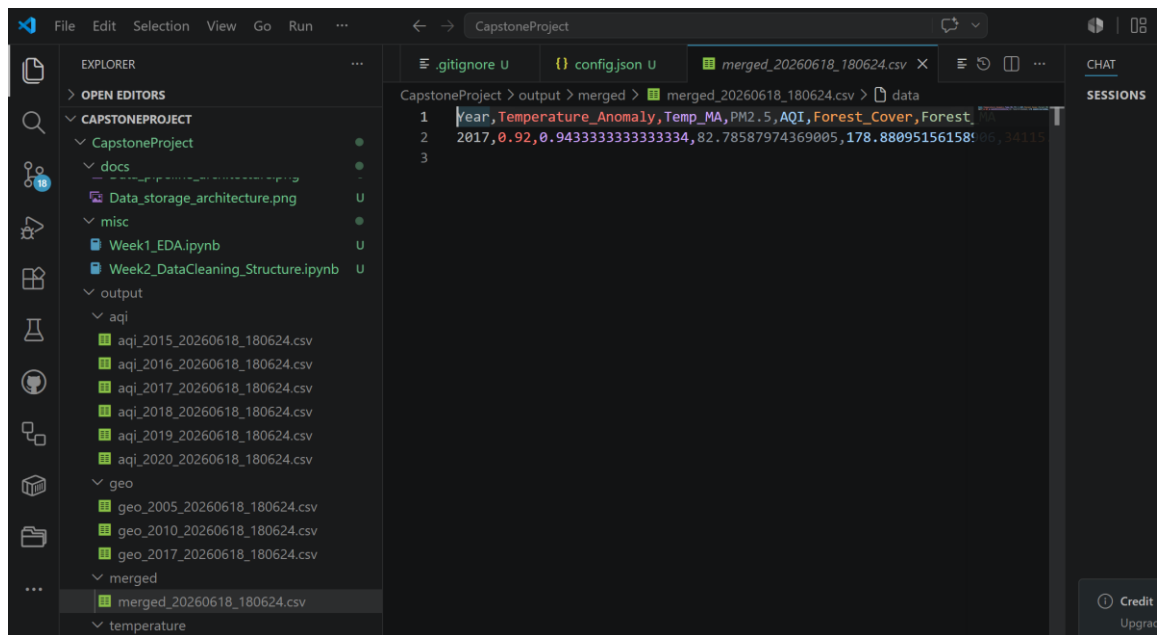
This screenshot displays the output folder structure generated by the pipeline. The data is organized into separate folders such as temperature, aqi, geo, and merged, representing a structured data lake architecture.

Figure 3: Partitioned Files



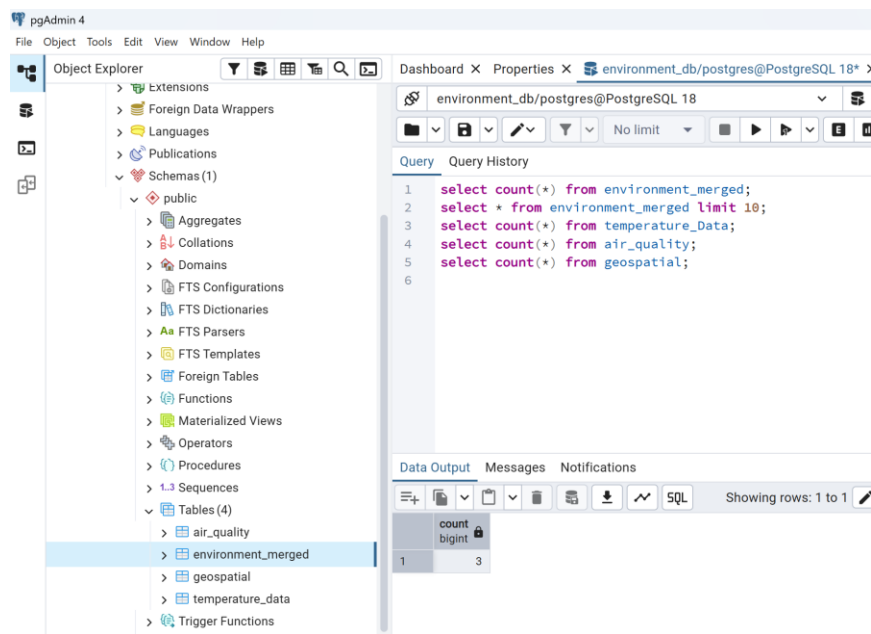
This screenshot shows the partitioned dataset files stored by year, such as aqi_2015_*.csv. This demonstrates time-based partitioning, which improves data organization and retrieval efficiency.

Figure 4: Merged Dataset



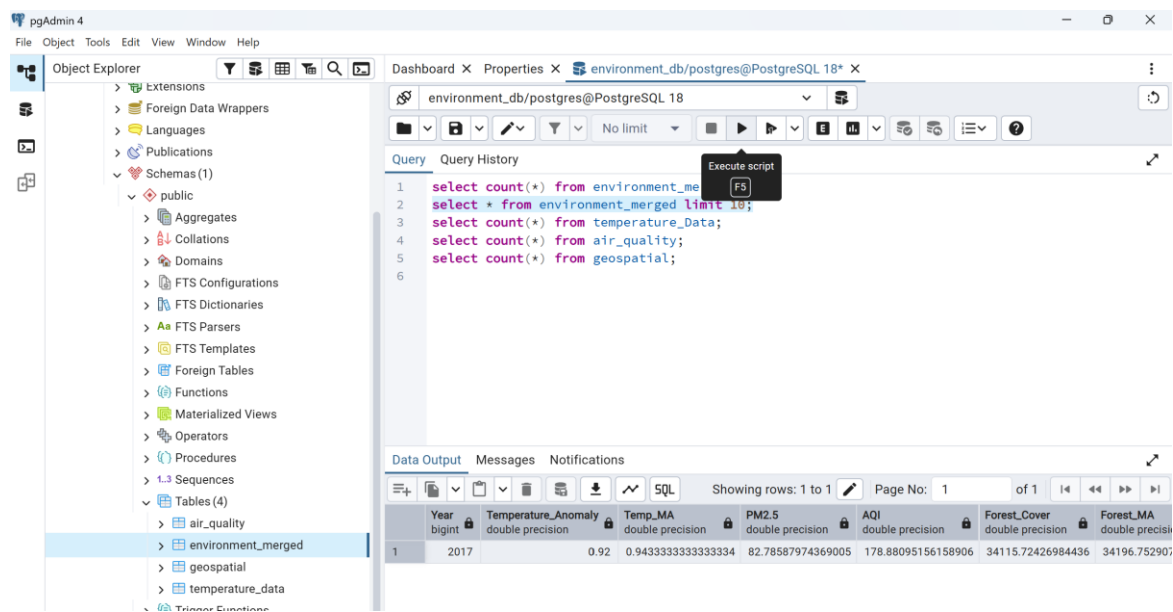
This screenshot shows the final merged dataset, which integrates temperature, air quality, and geospatial data. This dataset is used as the curated dataset for further analysis.

Figure 5: PostgreSQL Tables

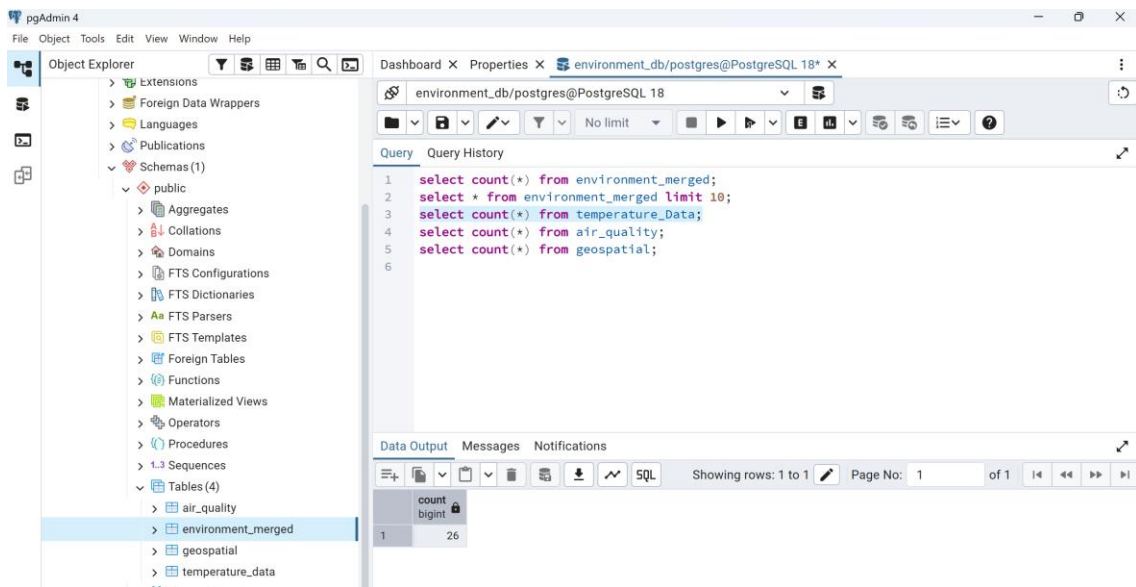


This screenshot shows the PostgreSQL database tables created by the pipeline, including temperature_data, air_quality, geospatial, and environment_merged. It confirms successful database integration.

Figure 6: SQL Query Results(Sample Data from Database)

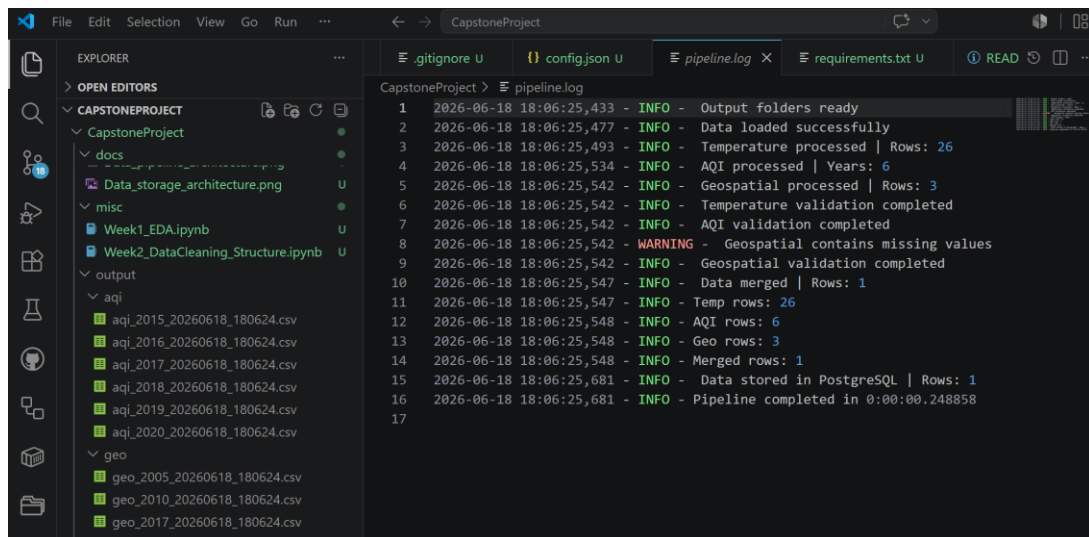


This screenshot shows the result of the SQL query "SELECT * FROM environment_merged;", confirming that data has been successfully inserted into the database.



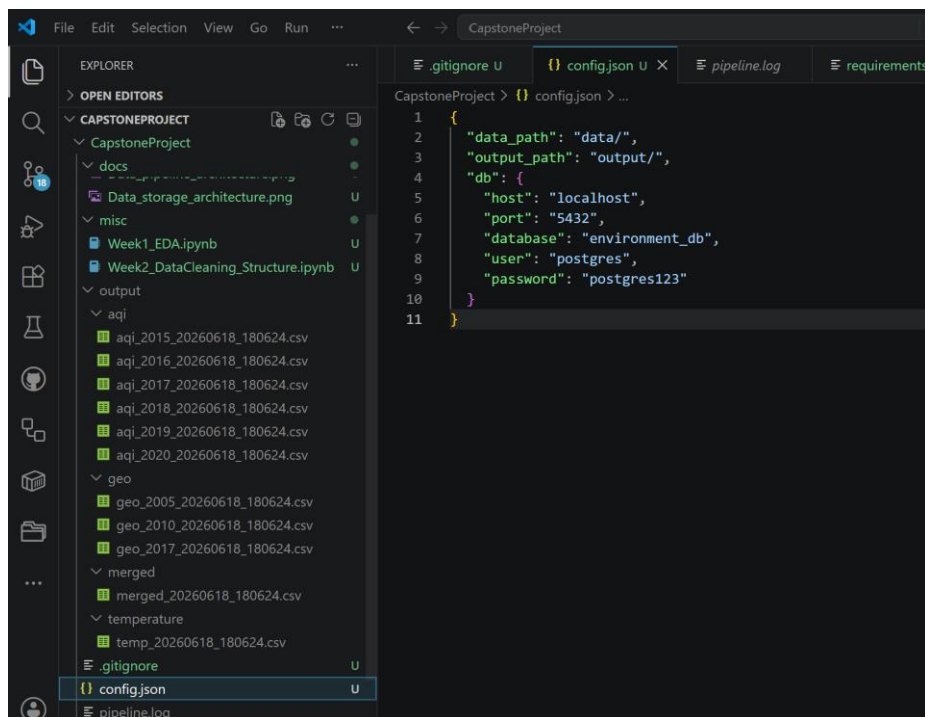
This screenshot displays sample records from the `environment_merged` table, confirming that the data is correctly stored and structured for analysis.

Figure 7: Pipeline Log File



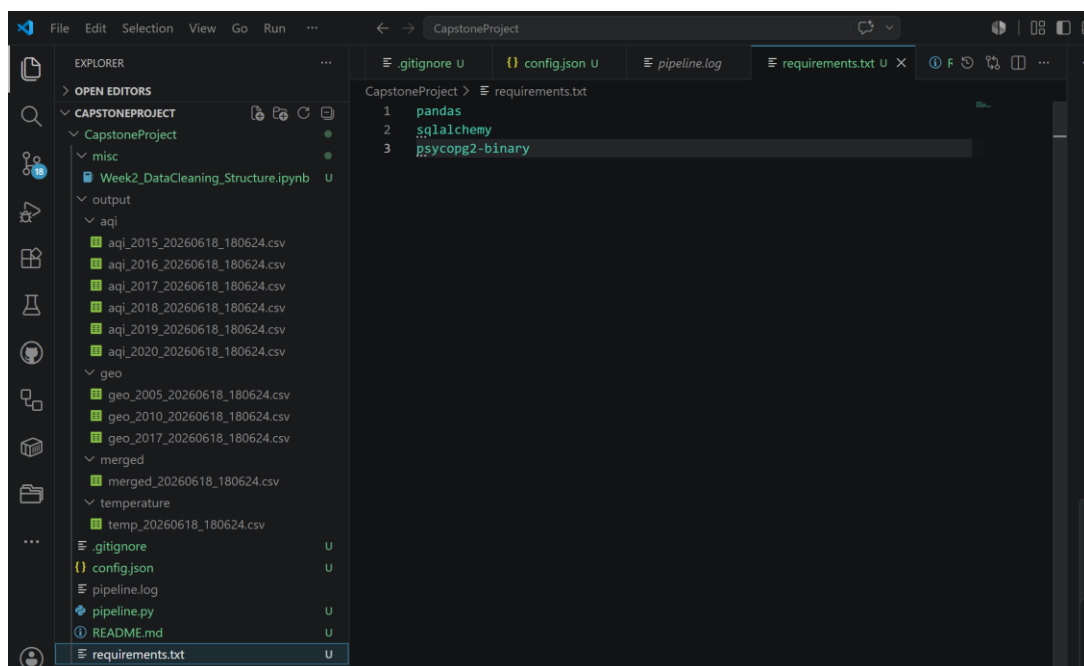
This screenshot shows the `pipeline.log` file, which records execution steps, processing details, and status messages for monitoring and debugging purposes.

Figure 8: Config File



This screenshot shows the configuration file used in the pipeline, which defines input paths, output paths, and database connection details for flexible and reusable execution.

Figure 9: Requirements File



This screenshot shows the requirements.txt file listing all required Python libraries. It enables easy setup of the project environment in a single command.